# Addressing Modes

- Various ways that the CPU of the AVR can access data
- determined when the microcontroller is organized

AVR - 13 distinct addressing modes
4 categories:
1) Single-register; immediate values
2) registers (2) GPR
3) direct address
4) register indirect

## single-register immediate mode

```
NEG R16
COM R16
INC R22
ROR R24
```
↑ single register

```
LDI R16, 0x24
SUBI R16, 0x7F
ANDI R20, 0x6B
```
↑ immediate value (not an address)

## register addressing mode (2 registers) (GPRs)

```
AND R21, R22
AND R24, R26
```
↑ ↑ GPRs

```
ADD R1, R2
MOV R0, R17
```

5. **(10 Points)** Given the addressing modes below, list **the** addressing mode used in each of the following instructions based on the operand given in **bold**.

Single-Register (Immediate)  Register  Direct  Register Indirect

a) LDI **R21**, 0x41 _____SINGLE REGISTER_____

b) STS **0x101**, R19 ____DIRECT_____

c) LD R20**, X+** _____INDIRECT_____

d) LPM R21, **Z** _____INDIRECT_____

e) CALL **DELAY** _____DIRECT_____

6. (**10 Points**) Specify whether the operand(s) of each instruction is(are) accessing (reading or storing) data in data memory (SRAM), code memory, or both. For each blank below, specify DATA, CODE, or BOTH.

a) LDI R21, 0x41 _____BOTH_____

b) STS 0x101, R19 _____DATA_____

c) LD R20, X+ _____DATA_____

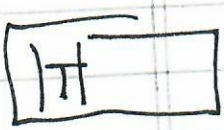
d) LPM R21, Z _____BOTH_____

e) CALL DELAY _____CODE_____

~~Store Table (6-7) (p. 6-6)~~

- single bit oriented instructions for AVR

C, Z, H, V, N, S



SReg (status register) - T bit is a temporary storage bit.

copy a bit {
- BST R16,4 ; Store bit 4 from R16 in T
- BLD R16,2 ; load bit T into bit 2 of (copy) R16

manipulate bits {
- SBI A,b ; set bit b in I/O register
- CBI A,b ; clear " " " "
- SBIC A,b ; skip next instruction if bi in I/O register is cleared
- SBIS A,b ; skip next instruction if b b in I/O register is set

check a bit {
- SBRC Rr,b ; skip next instruction i bit b in Rr is cleare
- SBRS Rr,b ; skip next instruction i bit b in Rr is set

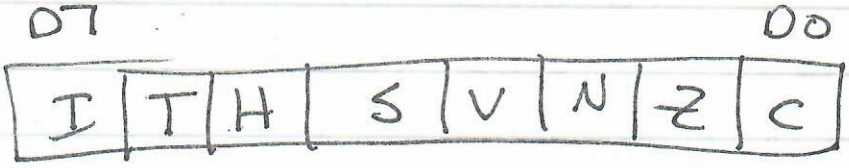
check a flag bit in SReg {
- BRBS s,k ; Branch if bit s in status register (SREG) is set ($-64 \le k \le 6$ of PC) $PC_{new} = PC_{old} + k + 1$
- BRBC s,k ; Branch if bit s in SReg is cleared ($-64 \le k \le 63$) of PC

```
; SREG - Status Register
.equ SREG_C = 0 ; Carry Flag
.equ SREG_Z = 1 ; Zero Flag
.equ SREG_N = 2 ; Negative Flag
.equ SREG_V = 3 ; Two's Complement Overflow Flag
.equ SREG_S = 4 ; Sign Bit
.equ SREG_H = 5 ; Half Carry Flag
.equ SREG_T = 6 ; Bit Copy Storage
.equ SREG_I = 7 ; Global Interrupt Enable
```

Ex 6-22 (p.228)
SBRS,
CBI instructions

see back

SReg




C = carry flag
Z = zero flag
N = negative flag
V = overflow flag
S = sign flag = N⊕V
H = Half carry flag (auxiliary carry)
T = temporary bit copy storage
I = Global Interrupt Enable

Conditional branch instructions

| | | |
|---|---|---|
| | BRCS branch if C=1 | BRCC branch if |
| SUB compare ⇒ | BRLO branch if C=0 | BRSH branch if C= |
| | BREQ branch if Z=1 | BRNE branch if Z= |
| | BRMI branch if N=1 | BRPL branch if N= |
| | BRVS branch if V=1 | BRVC branch if V= |
| | BRLT branch if S=1 | BRGE branch if S |
| | BRHS branch if H=1 | BRHC branch if H |
| | BRTS branch if T=1 | BRTC branch if T |
| | BRIE branch if I=1 | BRID branch if I |

# manipulating a bit instructions

BSET s ; flag bit set
$0 \le s \le 7 \Rightarrow$ bit in SReg

↳ BSET 0 ; set bit 0 ⇒ carry flag

BCLR s ; flag bit clear in SReg

SREG

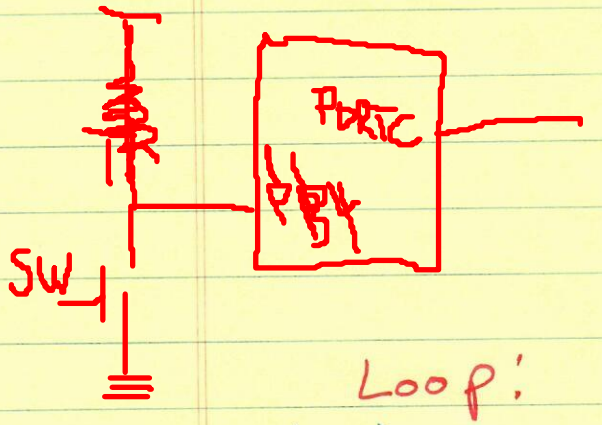
| | | | |
|---|---|---|---|
| → SEC | set carry C=1 | CLC | clear carry C=0 |
| SEZ | set zero Z=1 | CLZ | clear zero Z=0 |
| SEN | " negative N=1 | CLN | " negative N |
| SEV | " overflow V=1 | CLV | " overflow V |
| SES | " sign S=1 | CLS | " sign S=0 |
| SEH | " halfcarry H=1 | CLH | " halfcarry H |
| SET | " Temporary T=1 | CLT | " Temporary T= |
| SEI | " interrupt (enable) I=1 | CLI | " interrupt I (disable) |
| CLC | C=0 | | |

(ch 6.1 - 6.5 only)

A switch (SW) is connected to pin PB4. Write a program to check the status of the switch (PB4) and perform the following:

if SW=0, send 'N' to PORTC

⇒ if SW=1, send 'Y' to PORTC




```
.ORG 0
LDI R16, 0XFF
OUT DDRC, R16 ; all bits PC output

CBI DDRB, 4
SBI PORTB, 4  ; PB4 input, pull up

Loop: SBIS PINB, 4 ; PINB.4 = 1 ?
(SW=0) RJMP SEND_N ; PINB.4 = 0
LDI R18, 'Y' ; PINB.4 = 1 ⇒ 'Y'
OUT PORTC, R18 ; PC = 'Y' ASCII to PC
RJMP Loop ; continue polling

SEND_N: LDI R18, 'N' ; SW=0
OUT PORTC, R18 ; PC ⇒ 'N'
RJMP Loop ; continue polling
```

# Ch. 6

## Bit operators

| | | |
|---|---|---|
| % | modulo | |
| & | bitwise | AND |
| \| | " | OR |
| ^ | " | XOR |
| ~ | " | NOT |

<< shift left the expression by the number of places on the right of the operation

LDI R16, 0b00000011<<3

00000011

3 pos

R16 = 00011000

LDI R16, 1<<PB4

00000001

76543210

00010000 ⇒ R16

4

>> shift right expression

LDI R16, 0b110>>4

0b110=>00000110

00000110>>4 = 00000000

R16 = 0x00



# BCD & ASCII Conversion

| | | | |
|---|---|---|---|
| 0 | 0000 | 5 | 0101 |
| 1 | 0001 | 6 | 0110 |
| 2 | 0010 | 7 | 0111 |
| 3 | 0011 | 8 | 1000 |
| 4 | 0100 | 9 | 1001 |

## Unpacked BCD

- lower 4 bits represents the BCD value

0x04 ⇒ ~~0x00~~ 0100
4

## packed BCD

byte — upper nibble (BCD) | lower nibble (BCD)

0x47 ⇒ 0100 0111

Real Time clock ⇒ sec, time, day, month, year

---

SWAP Rd ; swap nibbles

```
LDI R16, 0x48
SWAP R16 ; R16= 0x84
```